

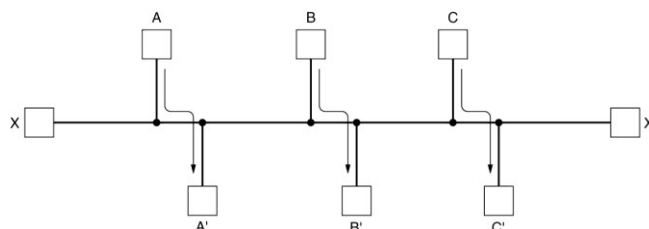
05 The Network Layer

05.05 Routing algorithms

Routing Algorithms

- Types of algorithms:
 - Static routing (non adaptive)
 - Adaptive routing (takes into account current network workload)
- The Optimality Principle
- Shortest Path Routing
- Flooding
- Distance Vector Routing
- Link State Routing
- Hierarchical Routing
- Broadcast Routing
- Multicast Routing

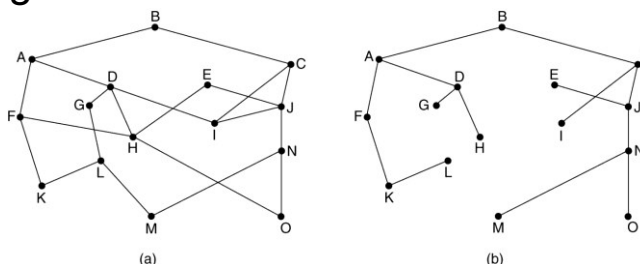
Routing Algorithms (2)



- Conflict between fairness and optimality
- If vertical traffic saturates horizontal lines it could be optimal to switch off x-x', but this wouldn't probably be fair
- Furthermore in what sense do we mean optimality? For instance:
 - Max network workload
 - Min average packet delay
 - But this 2 parameters are in conflict in typical queuing systems

The Optimality Principle

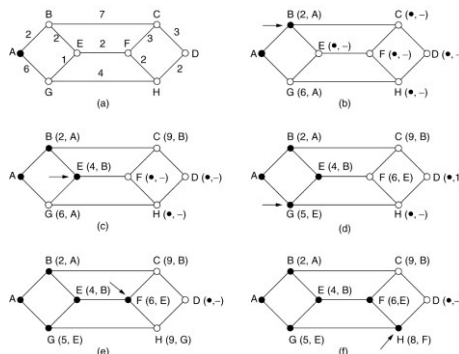
- If J is on the optimal path from I to K ($r_1=IJ$, $r_2=JK$) then the optimal from J to K is r_2
- The optimal paths from all possible sources to a given destination form a tree structure



(a) A subnet (b) A sink tree for router B.

Shortest Path Routing

- Weights are determined by possible metrics (even combination of these):
 - Distance, bandwidth, average traffic, communication cost, queue length, delay, etc.
- Dijkstra algorithm computes the shortest path in a weighted graph



- The first 5 steps used in computing the shortest path from A to D. The arrows indicate the working node

Dijkstra's algorithm

```

#define MAX_NODES 1024          /* maximum number of nodes */
#define INFINITY 1000000000     /* a number larger than every maximum path */
int n, dist[MAX_NODES][MAX_NODES]; /* dist[i][j] is the distance from i to j */

void shortest_path(int s, int t, int path[])
{
    struct state {               /* the path being worked on */
        int predecessor;        /* previous node */
        int length;              /* length from source to this node */
        enum {permanent, tentative} label; /* label state */
    } state[MAX_NODES];

    int i, k, min;
    struct state *p;

    for (p = &state[0]; p < &state[n]; p++) { /* initialize state */
        p->predecessor = -1;
        p->length = INFINITY;
        p->label = tentative;
    }
    state[t].length = 0; state[t].label = permanent;
    k = t; /* k is the initial working node */

```

Dijkstra's algorithm (2)

```

do {
    for (i = 0; i < n; i++)          /* Is there a better path from k? */
        if (dist[k][i] != 0 && state[i].label == tentative) /* this graph has n nodes */
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
    }

    /* Find the tentatively labeled node with the smallest label. */
    k = 0; min = INFINITY;
    for (i = 0; i < n; i++)
        if (state[i].label == tentative && state[i].length < min) {
            min = state[i].length;
            k = i;
        }
    state[k].label = permanent;
} while (k != s);

/* Copy the path into the output array. */
i = 0; k = s;
do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}

```

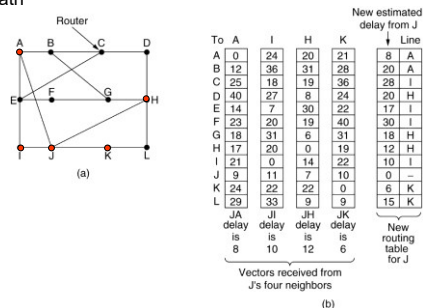
Flooding algorithm

- Flooding is a possible way of performing static routing: every packet is forwarded along all possible output lines
- This generates massively packets duplicates: to mitigate this fact counters can be added in the header of the packets (to be decreased at each hop) or they can be traced by routers to avoid re-transmissions
- A possible variant is selective (e.g. geographical) flooding
- Although not very practical in many contexts, flooding has its own practical uses:
 - Military applications (redundancy vs. fault tolerance)
 - Distributed databases
 - Comparison metric with other algorithms: since it chooses every possible parallel path it minimizes delay

Distance Vector Routing

(distributed Bellman-Ford or Ford-Fulkerson algorithm)

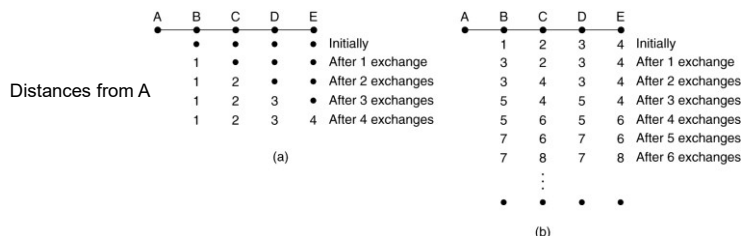
- The original algorithm of ARPANET used also in the Internet (with the name RIP: *Routing Information Protocol*)
- Routing table indexed by all other possible routers of the (sub)network
- Each record of the table is composed of:
 - Preferred line for that destination
 - Time or distance estimate associated to the destination
 - Distance can be evaluated in hops, time via special ECHO packets
 - Every T seconds each router sends its list of delay times to its neighbors
 - Example: J to G path



(a) A subnet. (b) Input from A, I, H, K, and the new routing table for J.

Distance Vector Routing

- Algorithm weakness: slow convergence
 - updates are rapid when delays switch from high to small values, but are slow in inverse cases



The count-to-infinity problem.

Distance Vector Routing Protocols

- **Examples of Distance Vector routing protocols:**
 - **Routing Information Protocol (RIP)**
 - **Interior Gateway Routing Protocol (IGRP)**
 - **Enhanced Interior Gateway Routing Protocol (EIGRP)**

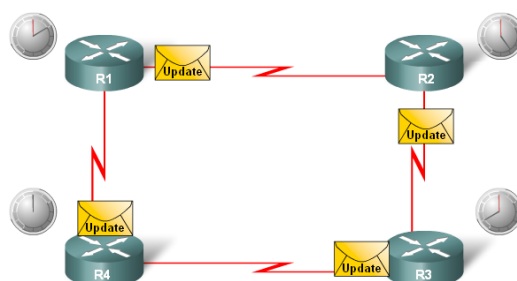
Distance Vector Routing Protocols

- **Distance Vector Technology**
 - **The Meaning of Distance Vector:**
 - A router using distance vector routing protocols knows 2 things:
 - **Distance** to final destination
 - **Vector, or direction**, traffic should be directed

Distance Vector Routing Protocols

Characteristics of Distance Vector routing protocols:

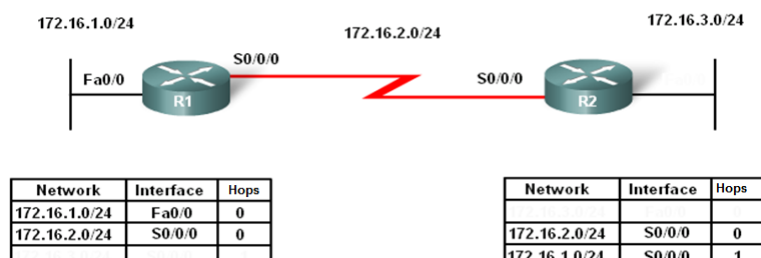
- Periodic updates
- Neighbors
- Broadcast updates
- Entire routing table is included with routing update



Distance Vector Routing Protocols

Purpose of Routing Algorithms

1. Send and Receive Updates
2. Calculate best path; install routes
3. Detect and react to topology changes



Distance Vector Routing Protocols

Routing Protocol Characteristics

–Criteria used to compare routing protocols

includes

- Time to convergence
- Scalability
- Resource usage
- Implementation & maintenance

Distance Vector Routing Protocols

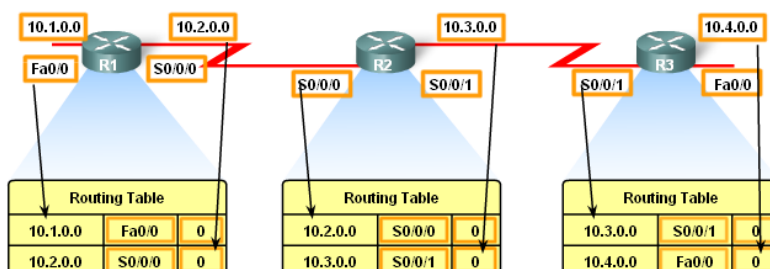
Advantages & Disadvantages of Distance Vector Routing Protocols

Advantages:	Disadvantages:
Simple implementation and maintenance. The level of knowledge required to deploy and later maintain a network with distance vector protocol is not high.	Slow convergence. The use of periodic updates can cause slower convergence. Even if some advanced techniques are used, like triggered updates which are discussed later, the overall convergence is still slower compared to link state routing protocols.
Low resource requirements. Distance vector protocols typically do not need large amounts of memory to store the information. Nor do they require a powerful CPU. Depending of the network size and the IP addressing implemented they also typically do not require a high level of link bandwidth to send routing updates. However, this can become an issue if you deploy a distance vector protocol in a large network.	Limited scalability. Slow convergence may limit the size of the network because larger networks require more time to propagate routing information.
	Routing loops. Routing loops can occur when inconsistent routing tables are not updated due to slow convergence in a changing network.

Network Discovery

- **Router initial start up (Cold Starts)**
 - Initial network discovery
 - ▀ Directly connected networks are initially placed in routing table

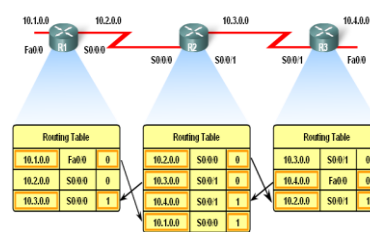
Network Discovery - Cold Start



Network Discovery

- **Initial Exchange of Routing Information**
 - If a routing protocol is configured then
 - Routers will exchange routing information
- Routing updates received from other routers
 - Router checks update for new information
 - ▀ If there is new information:
 - Metric is updated
 - New information is stored in routing table

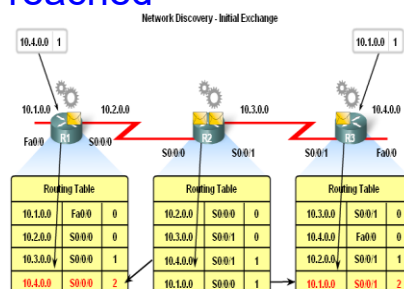
Network Discovery - Initial Exchange



Routers update routing tables with new information

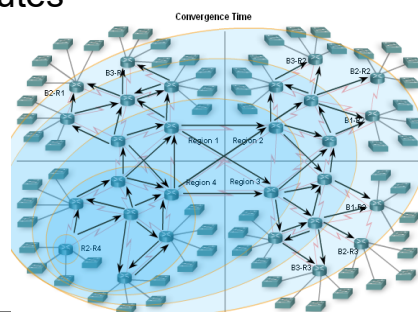
Network Discovery

- **Exchange of Routing Information**
 - Router convergence is reached when
 - All routing tables in the network contain the same network information
 - Routers continue to exchange routing information
 - If no new information is found then Convergence is reached



Network Discovery

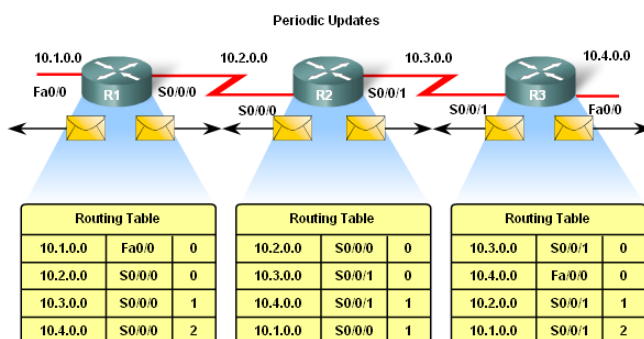
- **Convergence must be reached** before a network is considered completely operable
- Speed of achieving convergence consists of 2 interdependent categories
 - Speed of broadcasting routing information
 - Speed of calculating routes



Routing Table Maintenance

• Periodic Updates: RIPv1 & RIPv2

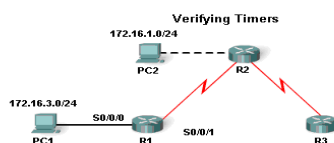
These are **time intervals** in which a router sends out its entire routing table.



Routing Table Maintenance

• RIP uses 4 timers

- Update timer
- Invalid timer
- Holddown timer
- Flush timer



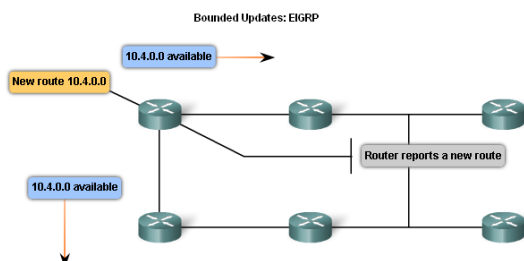
```
R1#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

172.16.0.0/24 is subnetted, 3 subnets
R    172.16.1.0 [120/1] via 172.16.2.2, 00:00:18, Serial0/0/0
C    172.16.2.0 is directly connected, Serial0/0/0
C    172.16.3.0 is directly connected, FastEthernet0/0
R    192.168.1.0/24 [120/1] via 192.168.3.1, 00:00:27, Serial0/0/1
R    192.168.1.0/24 [120/1] via 172.16.2.2, 00:00:18, Serial0/0/0
C    192.168.3.0/24 is directly connected, Serial0/0/1
R1#
```

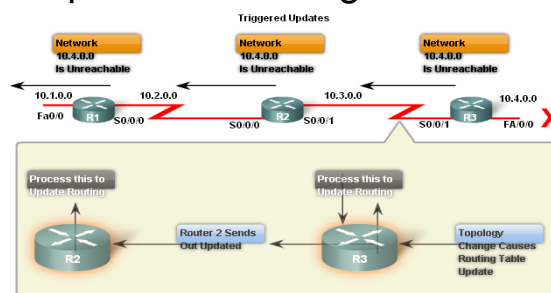
Routing Table Maintenance

- **Bounded Updates: EIGRP**
- EIGRP routing updates are
 - Partial updates
 - Triggered by topology changes
 - Bounded
 - Non periodic



Routing Table Maintenance

- **Triggered Updates**
 - Conditions in which triggered updates are sent
 - Interface changes state
 - Route becomes unreachable
 - Route is placed in routing table



Routing Table Maintenance

• Random Jitter

–Synchronized updates

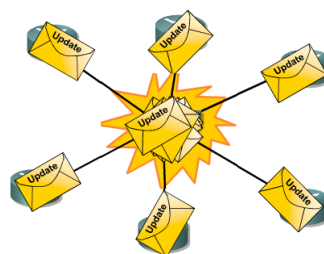
- A condition where multiple routers on multi access LAN segments transmit routing updates at the same time.

■ Problems with synchronized updates

- Bandwidth consumption
- Packet collisions

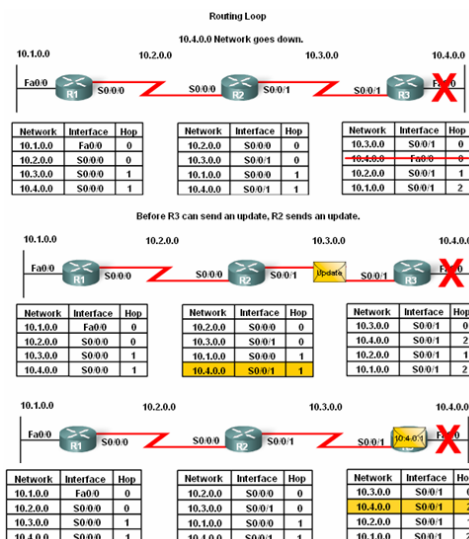
■ Solution to problems with synchronized updates

- Use of random variable called RIP_JITTER



Routing Loops

- **Routing loops** are conditions in which **a packet is continuously transmitted** within a series of routers without ever reaching its destination.



Routing Loops

- **Routing loops** may be **caused by**:
 - Incorrectly configured static routes
 - Incorrectly configured route redistribution
 - Slow convergence
 - Incorrectly configured discard routes
- **Routing loops** can **create the following issues**
 - Excess use of bandwidth
 - CPU resources may be strained
 - Network convergence is degraded
 - Routing updates may be lost or not processed in a timely manner

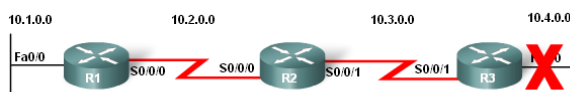
Routing Loops

Count to Infinity

This is a routing loop whereby packets bounce infinitely around a network.

Count to Infinity

Each round of updates continues to increase hop count.



Network	Interface	Hop
10.1.0.0	Fa0/0	0
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/0	1
10.4.0.0	S0/0/0	24

Network	Interface	Hop
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/1	0
10.1.0.0	S0/0/1	1
10.4.0.0	S0/0/1	23

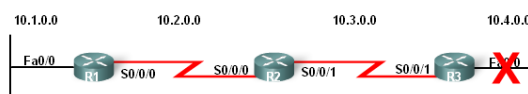
Network	Interface	Hop
10.3.0.0	S0/0/1	0
10.4.0.0	S0/0/1	22
10.2.0.0	S0/0/1	1
10.1.0.0	S0/0/1	2

Routing Loops

- Setting a maximum
- **Distance Vector routing protocols** set a specified metric value to indicate infinity

Once a router “counts to infinity” it marks the route as unreachable

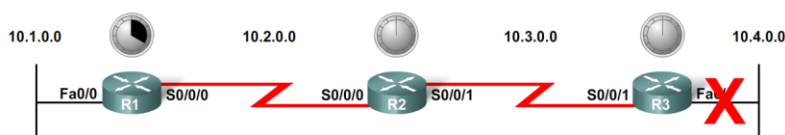
10.4.0.0 is unreachable. Hop count is 16.



Network	Interface	Hop	Network	Interface	Hop	Network	Interface	Hop
10.1.0.0	Fa0/0	0	10.2.0.0	S0/0/0	0	10.3.0.0	S0/0/1	0
10.2.0.0	S0/0/0	0	10.3.0.0	S0/0/1	0	10.4.0.0	S0/0/1	16
10.3.0.0	S0/0/0	1	10.1.0.0	S0/0/0	1	10.2.0.0	S0/0/1	1
10.4.0.0	S0/0/0	16	10.4.0.0	S0/0/1	16	10.1.0.0	S0/0/1	2

Routing Loops

- Preventing loops with holddown timers
 - **Holddown timers** allow a router to not accept any changes to a route for a specified period of time.
 - **Point of using holddown timers**
 - Allows routing updates to propagate through network with the most current information.



Network	Interface	Hop
10.1.0.0	Fa0/0	0
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/0	1
10.4.0.0	S0/0/0	2

Network	Interface	Hop
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/1	0
10.1.0.0	S0/0/0	1
10.4.0.0	S0/0/1	1

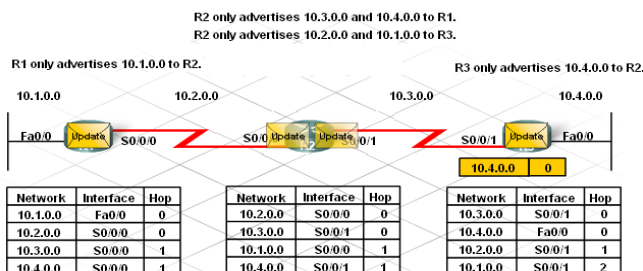
Network	Interface	Hop
10.3.0.0	S0/0/1	0
10.4.0.0	S0/0/1	0
10.2.0.0	S0/0/1	1
10.1.0.0	S0/0/1	2

Routing Loops

- The **Split Horizon Rule** is used to prevent routing loops
- Split Horizon rule:**

A router should not advertise a network through the interface from which the update came.

Split Horizon Rule for 10.4.0.0



Antonio Della Selva

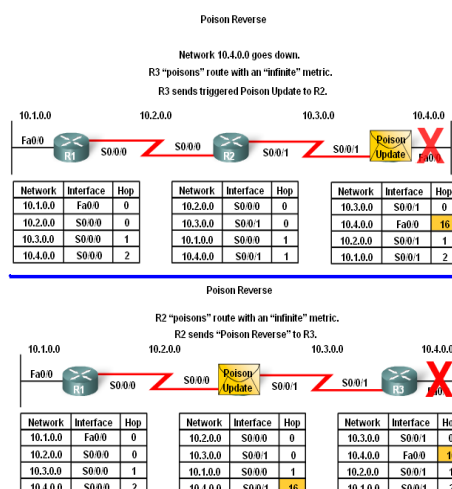
isti information science and technology institute

31/85

Routing Loops

- Split horizon with poison reverse**

The rule states that once a router learns of an unreachable route through an interface, advertise it as unreachable back through the same interface



Antonio Della Selva

isti information science and technology institute

32/85

Routing Loops

- IP & TTL

- Purpose of the TTL field

The TTL field is found in an IP header and is used to prevent packets from endlessly traveling on a network

- How the TTL field works

- TTL field contains a numeric value

The numeric value is decreased by one by every router on the route to the destination.

If numeric value reaches 0 then Packet is discarded.

Routing Protocols Today

- Factors used to determine whether to use RIP or EIGRP include

- Network size
 - Compatibility between models of routers
 - Administrative knowledge

Distance Vector Routing Protocols Compared

	Ripv1	Ripv2	IGRP	EIGRP
Speed of Convergence	Slow	Slow	Slow	Fast
Scalability – size of network	Small	Small	Small	Large
Use of VLSM	No	Yes	No	Yes
Resource usage	Low	Low	Low	Medium
Implementation and maintenance	Simple	Simple	Simple	Complex

Routing Protocols Today

- **RIP**

- **Features of RIP:**

- Supports split horizon & split horizon with poison reverse
 - Capable of load balancing
 - Easy to configure
 - Works in a multi vendor router environment

Routing Protocols Today

- **EIGRP**

- **Features of EIGRP:**

- Triggered updates
 - EIGRP hello protocol used to establish neighbor adjacencies
 - Use of topology table to maintain all routes
 - Classless distance vector routing protocol
 - Cisco proprietary protocol

Link State Routing

- Also called shortest path first (SPF) forwarding
 - Named after Dijkstra's algorithm (1959) which it uses to compute routes
- All routers have tables which contain a representation of the entire network topology
 - In the form of lists of routers and information about each router's neighbours and the connection between the two

Link State Routing

- Each router creates a *link state packet* (LSP) which contains names (e.g. network addresses) and cost to each of its neighbours
 - The LSP is transmitted to *all* other routers, who each update their own records
 - When a routers receives LSPs from all routers, it can use (collectively) that information to make topology-level decisions

Link State Packets

- LSPs are generated and distributed when:
 - A time period passes
 - New neighbours connect to the router
 - The link cost of a neighbour has changed
 - A link to a neighbour has failed (link failure)
 - A neighbour has failed (node failure)

Link State Packets

- LSP are essentially a list of tuples, containing:
 - The name of a neighbour to a router
 - Which may be a router or a network
 - The cost of the link to that neighbour

Link State Packets

- Distribution of LSPs can be difficult
 - Routers themselves are the means for delivering messages
 - How do routers deliver their own messages, particularly when routers are in an inconsistent state
 - e.g. During link failure, before each router has been notified of the problem

Link State Packets

- One method for LSP distribution: Flooding
 - Each LSP received is transmitted to every direct neighbour (except the neighbour where the LSP came from)
 - This creates an exponential number of packets on the network (similar to $O(2^R)$, where R is the number of routers)
 - It does, however, guarantee that the LSP will be received by every router
 - Assuming that node or link failure does not occur, and LSPs are not somehow lost

Link State Packets

- An improvement on this scheme is as follows:
- When an LSP is received, it is compared with the stored copy
 - If it is identical to the stored copy, it is dropped
 - If it is different, the stored LSP is overwritten with the new LSP and the LSP is transmitted to every direct neighbour (except the source of the LSP)
- This scheme works because if a given router has already received a LSP from another neighbour, it will have also already distributed the LSP to all of its neighbours

Link State Routing Algorithm

- Ok, now that we know how to distribute LSPs, how are they used to determine routes?
 - The algorithm used (mostly) was developed by Dijkstra
 - Essentially, the algorithm runs at each router, computing each possible path to the destination, adding up each cost
 - The path with the lowest cost is used

Link State Routing Algorithm

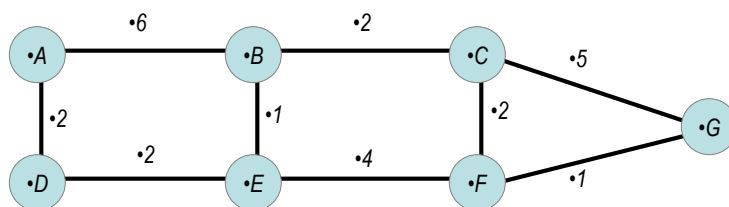
- The algorithm requires the following information:
 - Link state database: List of all the latest LSPs from each router on the network
 - Path: Tree structure storing previously computed best paths
 - Consider this a sort of cache
 - Data type for nodes: (ID, path cost, port)
 - Tent: Tree structure storing paths currently being tested and compared (tentative)
 - Consider this a sort of rough workspace
 - Data type for nodes: (ID, path cost, port)
 - Forwarding database: Table storing all IDs that can be reached, and the port to which messages should be sent
 - This is simply a reduced version of the 'Path', which contains (destination,port) pairs
 - This can be used by the router to quickly forward packets for which the best path has already been determined
 - Data type for table rows: (ID, port)

Dijkstra's LSR Algorithm

- Initially, PATH is just a root containing (this router's ID, 0, 0)
- For every node placed into path, N:
 - For all neighbours M of node N:
 - If M is not in TENT, add a node to TENT for M (use the LSP for N to determine link cost)
 - If M is in TENT already, and its cost is lower than an existing entry for M, replace that entry with information from N's LSP
 - If M is in TENT already, but its cost is higher, ignore N's link to M
 - Calculate the shortest route in TENT
 - If the shortest route has lower cost than the route in PATH, overwrite the route in PATH with the route in TENT

Dijkstra's LSR Algorithm

- Consider the following network:



• Link state database:

A	B	C	D	E	F	G
B 6	A 6	B 6	A 2	B 1	C 2	C 5
D 2	C 2	F 2	E 2	D 2	E 4	F 1
	E 1	G 5		F 4	G 1	

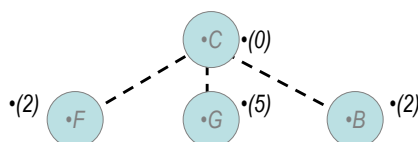
Dijkstra's LSR Algorithm

- Now, if we want to generate a PATH for C:
 - First, we add (C,0,0) to PATH



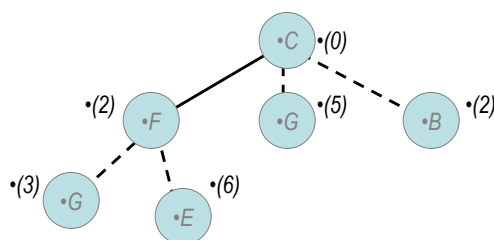
Dijkstra's LSR Algorithm

- Examine C's LSP
 - Add F, G, and B to TENT



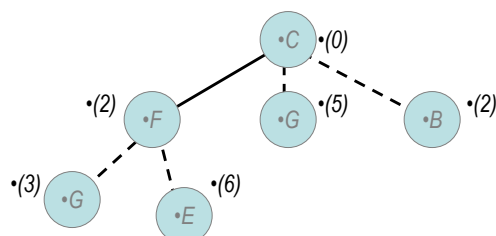
Dijkstra's LSR Algorithm

- Place F in PATH (shown as solid line)
 - Add G and E to TENT (adding costs)



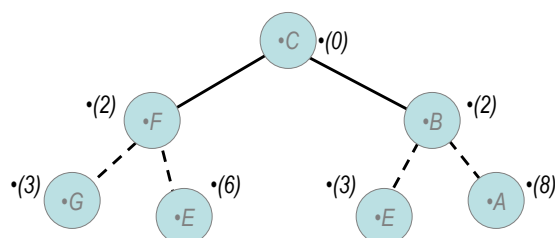
Dijkstra's LSR Algorithm

- G exists in TENT twice, keep only the best
 - The new G is a better path than the old ($3 < 5$)



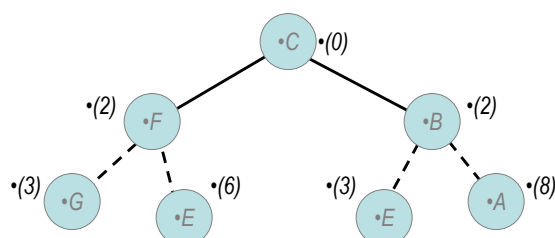
Dijkstra's LSR Algorithm

- Put B into path (shown as solid line)
 - Add A and E to TENT



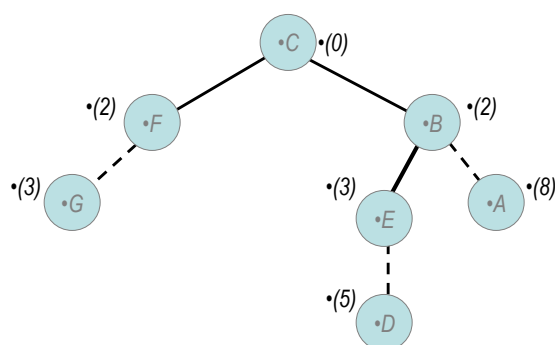
Dijkstra's LSR Algorithm

- E exists in TENT twice, keep only the best
 - The new E is better than the old ($3 < 6$)



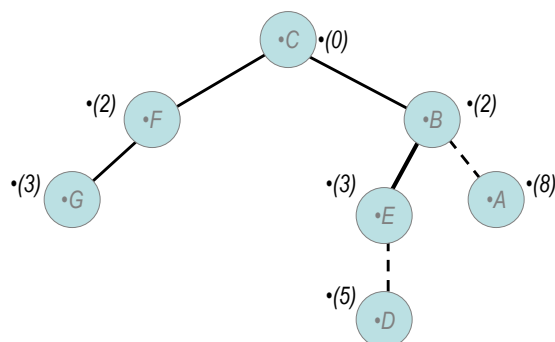
Dijkstra's LSR Algorithm

- Place E in PATH (shown as solid line)
 - Add D to TENT



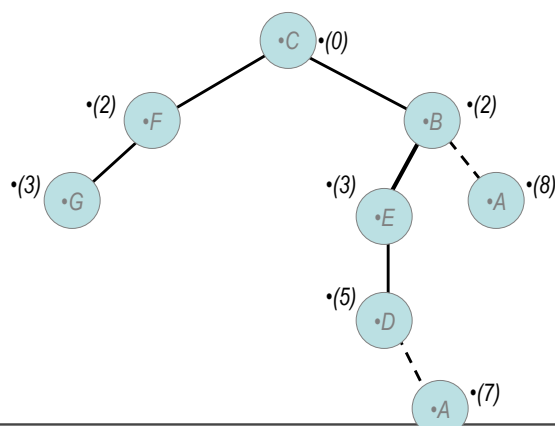
Dijkstra's LSR Algorithm

- Place G in PATH (shown as solid line)
 - All G's LSP elements already exist in TENT



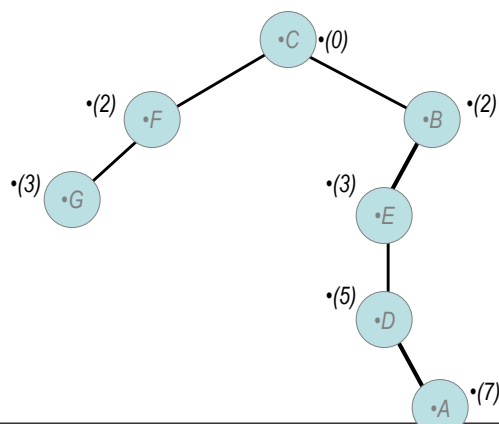
Dijkstra's LSR Algorithm

- Place D in PATH (shown as solid line)
 - Add path to A since it is better than old A



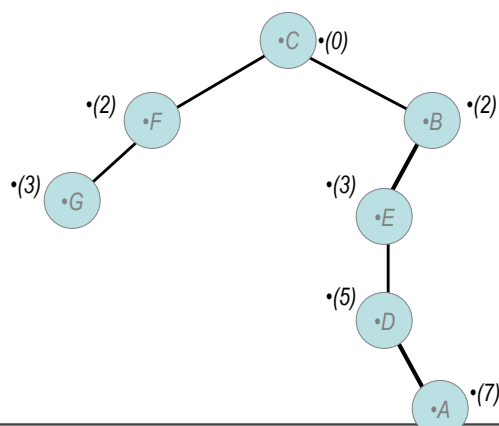
Dijkstra's LSR Algorithm

- Place A in PATH (shown as solid line)
 - All A's LSP elements already exist in PATH



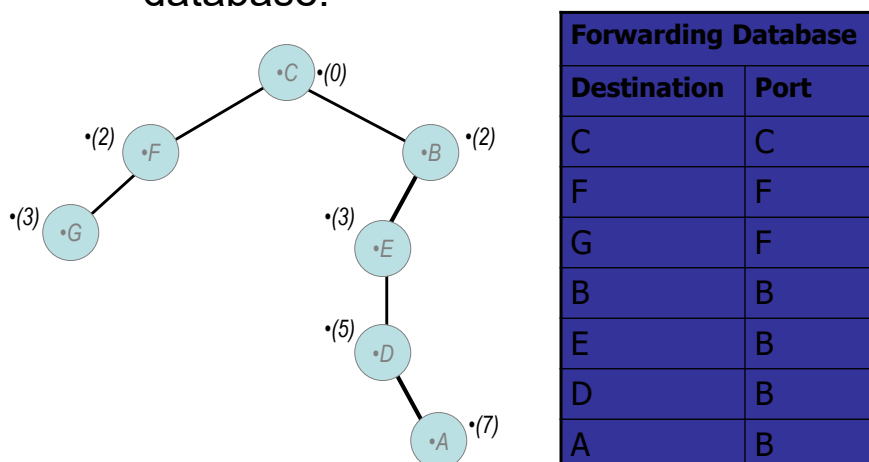
Dijkstra's LSR Algorithm

- We are done since all routes from TENT were placed into PATH



Dijkstra's LSR Algorithm

- We can now create a forwarding database:



LSR Topology Changes

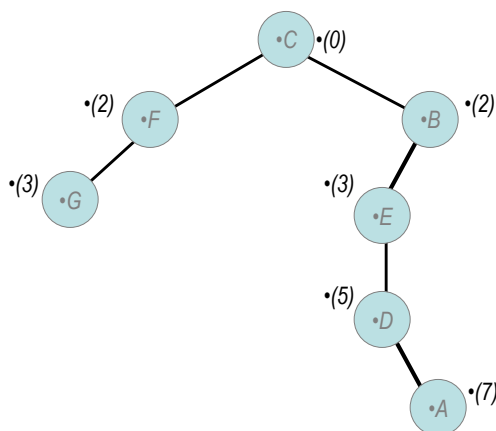
- LSR forwarding tables must be recalculated whenever a topology change occurs
 - For example, a new router and/or link is added to the network
 - This new link may provide a more efficient route to one or more other nodes
 - For example, a given link's cost is reduced
 - This new link may now provide the lowest total cost route to a destination that was previously forwarded in another direction
 - For example, a given link's cost is increased
 - This new link may no longer provide the lowest total cost route to a given destination, and another route should now be chosen

LSR Topology Changes

- In a nutshell, LSR routers should invalidate (indicate that it needs to be regenerated) its PATH data structure, and thus its forwarding table
 - The entire PATH generation algorithm (e.g. Dijkstra's algorithm) should be reapplied

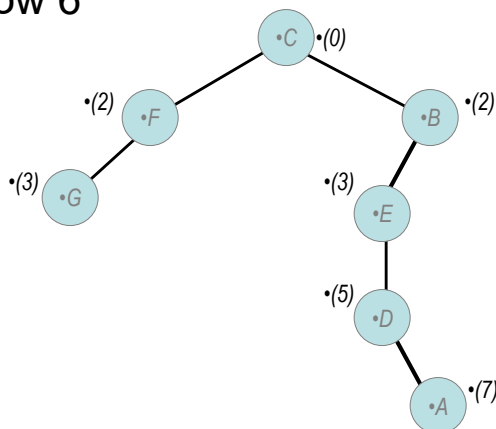
Topology Change Example

- Let's consider our previously generated PATH structure for the router C



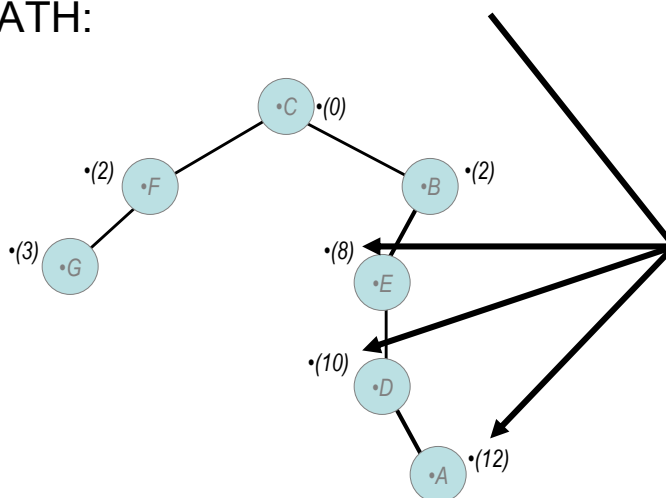
Topology Change Example

- Say we receive an LSP from router B, indicating the link cost from B to E is now 6



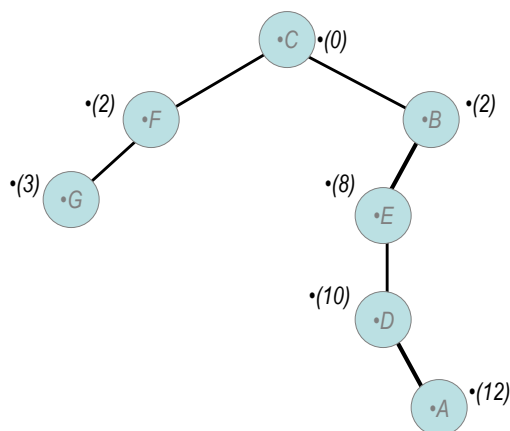
Topology Change Example

- The total route costs are different in PATH:



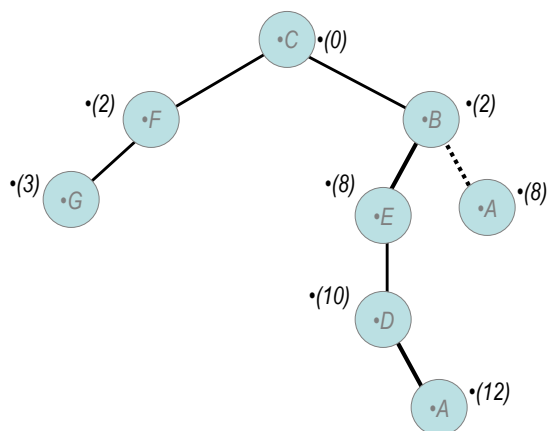
Topology Change Example

- Consider for now, only the cost to A



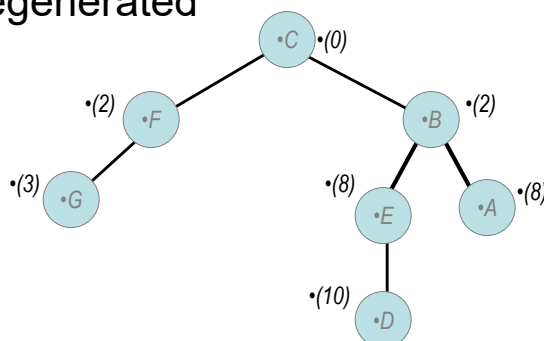
Topology Change Example

- Recall that another path to A existed
- Now, that path is more efficient



Topology Change Example

- The PATH data structure is complete, the forwarding table can now be regenerated



Topology Change Example

- In a router, which will be running as a computer program, finding if a new path exists essentially requires complete re-execution of Dijkstra's algorithm
- For example, there could have been many routes to A, each of which would have to be compared to find the most efficient route

Open Shortest Path First

OSPF

OSPF

- Open SPF protocol
 - SPF: Shortest path first
 - Essentially the OSPF specification is one specification describing an algorithm implementing shortest path forwarding
 - It is open, meaning anyone can implement the specification at no cost
- Since OSPF is a link state routing (LSR) protocol, it can have load balancing
 - However, to be deployed on large-scale WANs, the LSP propagation must be limited
 - OSPF partitions networks into regions called 'areas' which contain a subset of the routers
 - This is similar to schemes used in other LSR protocols

OSPF Autonomous Systems

- An OSPF AS allows a multi-level routing strategy to be employed
- The complete system is partitioned into autonomous systems (AS)
 - Within each AS, OSPF routing is used
 - In this way, the limitation of the number of routers in link state routing is not a factor
 - Messaging between AS can use ATM

OSPF Messages

- There are 5 types of messages in OSPF:
 - Hello messages
 - Allow routers to test if a node is reachable
 - Link State Advertisement (LSA)
 - Topology information from a router (i.e. LSPs)
 - Link status request (LSR)
 - Requests send to another router to determine the status of one or more links
 - Link status update (LSU)
 - Responses to a link status request message
 - Link status acknowledgement
 - Used to indicate that the LSU was received (reliable transfer)

OSPF Hello Packets

- When a router wants to test if a node is reachable, it sends a Hello packet
- If the node is reachable, it will respond with its own Hello packet
- Hello packets contain a list of reachable addresses (among other things)
 - A router might query the node about one of those addresses
 - The node will respond with topology (connectivity) information about the host at that address
 - In the form of an LSA

OSPF LSA Packets

- When requested, these packets contain a list of links (forming a complete route) to a destination
- This can be used (including the link cost information) to determine the 'shortest' path

OSPF Link Status Requests

- This packet represents a request for information about one or more links
- A router may be given this request if another router has outdated information

OSPF Link Status Updates

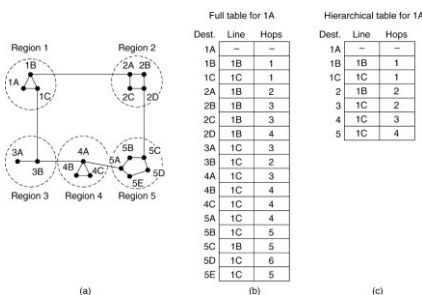
- This is a response to a link status request
- It contains information about each link requested
 - Most importantly: the length (cost) of the link

Multicast Routing in OSPF

- In OSPF and other link state implementations, multicast routing is supported
 - For a multicast message, Dijkstra's path tree is also created
 - However, in this case, the sender is the root of the tree, not the current router
 - The current router sends the message to *all* of its direct child nodes in the tree

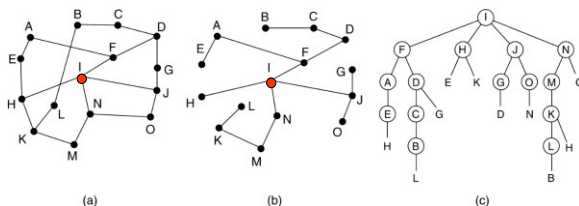
Hierarchical Routing

- Hierarchical routing is the solution for mitigating routing tables growth (that impacts memory usage, CPU time to analyze tables and bandwidth to transmit state information)
- Dividing routers into regions is a natural solution that trades off smaller routing tables for possibly longer paths
- For instance, in the following figure, the best path from 1A to 5C passes through region2 but all traffic to region 5 is sent through region 3 since most destinations in region 5 are better served by this solution
- How many levels do we need for optimal hierarchical clustering?
- Kamoun&Kleinrock (1979) demonstrated that:
 - A network composed of N routers has $\ln(N)$ optimal levels, giving $e \cdot \ln(N)$ records per router
 - The increasing in length of the average path is small enough to be considered acceptable



Broadcast Routing

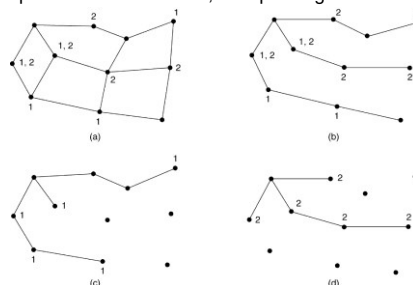
- Some applications may need broadcast (of course broadcast networks make use of special broadcast addresses as we saw for Ethernet LAN's). Several solutions may be used:
- Trivial solutions
 - Flooding
 - sending each packet to every possible destination
 - But... too much packets and bandwidth consumption
- Multidestination routing:
 - Each packet contains a list of all destinations. When a router forwards a packet along 2 or more lines only the destinations of the corresponding line are kept in the packet
- Spanning tree based
 - If a router knows a spanning tree (i.e. a set nodes that doesn't contain cycles) it copies a packet on all its lines belonging to the spanning tree
 - Good method for bandwidth and nr. of generated packets but routers do not always know spanning trees (yes for link state routing, no for distance vector)
- Reverse path forwarding
 - Used when routers ignore spanning trees
 - If a packet arrives through the line usually used to forward packets to the source of the broadcast message is probably the first copy (probably because it followed an optimal path). Otherwise, the packet is rejected (since, probably, it is a duplicate)



Reverse path forwarding. (a) A subnet. (b) a Sink tree. (c) The tree built by reverse path forwarding.

Multicast Routing

- Some applications may need to send messages to members of groups which are numerically significant, even if small if compared with the whole network (*multicast*). Point to point or broadcast solutions can be wasteful and/or insecure
- Groups need to be handled: hosts communicate to routers (or routers periodically check hosts connected to them) their group and then information flows along the network through router-router links
- Algorithm for multicast is based on spanning tree reduction (see fig. below):
 - If a link state routing is used, every router knows the whole network topology, hence spanning tree can be reduced starting from the end of each path and backtracking to the root, removing all routers outside a certain group
 - If distance vector routing is used a recursive procedure is adopted. It is based on PRUNE messages sent by routers that receive a multicast message and don't have any host for that group (disadvantage: if n groups of m members each, mn spanning trees need to be kept in memory)



(a) A network. (b) A spanning tree for the leftmost router. (c) A multicast tree for group 1. (d) A multicast tree for group 2.